Data Science Methods for Cognitive Science

Dan Hicks, hicks.daniel.j@gmail.com

Fri Aug 28 19:07:59 2020

Contents

Preface **2**

Setting up **2**
0.1 A Note on Accessibility . . . 3
0.2 Hardware Recommendations . . . 3
0.3 R and RStudio . . . 3
0.4 Slack . . . 4
0.5 GitHub . . . 4
0.6 Screen Layouts . . . 4
0.7 Pre-Class Survey . . . 5

1 Introduction **5**
1.1 Land Acknowledgement . . . 5
1.2 About me . . . 5
1.3 What this course isn't, and is . . . 6
1.4 Learning outcomes . . . 6
1.5 Workflow . . . 7
1.6 Notes on some course design decisions . . . 7

2 What is this thing called data science? **8**
2.1 Standard definition . . . 8
2.2 A prior question: What is this thing called "data"? . . . 8
2.3 Back to "data science" . . . 10

I Some fundamental tools **10**

3 Git and Version Control **10**
3.1 Some motivating examples . . . 10
3.2 Version control . . . 11
3.3 How to do things with git . . . 11
3.4 Working with GitHub remotes . . . 12
3.5 Moar git stuff . . . 13

4 R Basics **13**
4.1 Let's get started with RStudio . . . 13
4.2 Base R data types . . . 15
4.3 Examining variables . . . 16
4.4 Logical operators . . . 17
4.5 Missing data . . . 17

5 Functional Programming and Data Science **18**

6 Warnings, Errors, and Getting Help **18**
6.1 Recommended reading and watching . . . 18
6.2 Messages, warnings, and errors . . . 19
6.3 Where to go for help . . . 19
6.4 Writing a reproducible example: `reprex` . . . 19
6.5 Do not do these things in your reprex . . . 19

II Exploratory Data Analysis **20**

7 Waterfalls, Spirals, and Philosophy of Science **20**

8 Exploratory Data Analysis and the Ladder of Models **21**

III Cleaning Data **21**

IV Reproducibility **21**

V Case Study: The COMPAS Dataset **21**

Preface

This document is a set of notes (and perhaps later a textbook) for a course on data science methods for graduate students in social and behavioral sciences, taught by Professor Dan Hicks (they/them), UC Merced.

Setting up

Before the first day of class, you'll need to set up a few things: **R, RStudio, Slack, Zoom, and a GitHub account**. I'd also like you to fill out a **pre-class survey**. R is the primary piece of software we'll be learning to use. RStudio is an IDE (integrated development environment) for using R. (Think of R as the engine of a car and RStudio as the dashboard, steering wheel, and pedals.) Slack will be the main way we communicate; we'll use Zoom for class time. GitHub is a service for sharing and collaborating on software engineering projects.

Table 1: Key links for setup

R	https://cran.r-project.org/
RStudio	https://rstudio.com/products/rstudio/download/
Slack	http://datascienceme-ke13588.slack.com/
GitHub	https://github.com/; https://github.com/data-science-methods
Survey	https://ucmerced.az1.qualtrics.com/jfe/form/SV_eVU8JWREUu4UNH7

I'll assume you have R and RStudio installed using either *the cloud way* or *the Mac way*. If your computer is running Windows, Linux, or some other operating system that isn't Mac OS, I'm going to assume you have things set up the cloud way. These other operating systems are perfectly fine. But I don't use them, which makes it much harder for me to help you if things aren't working. And Windows handles things like dependencies, file paths, text file formats in idiosyncratic ways that can cause weird errors.

In either case, all of the software and services we'll be using are either free or freemium. You won't need any of the paid extras for this course.

0.1 A Note on Accessibility

I chose the tools and platforms for this course in part because they're industry-standard. If you pursue a career as a data scientist in industry, you'll be expected to use Slack and Github on a daily basis. And RStudio is by far the most commonly used IDE ("integrated development environment") for R.

However, like many other technologies, they were originally developed using ableist assumptions about "normal" computer users. In response to criticism, the developers of these systems and tools have gone back and made their technologies more accessible. But there may still be barriers to accessibility that I have not anticipated. And of course the pandemic has created additional barriers, such as dependency on screensharing.

If you encounter a barrier to participating in this course — even a small inconvenience — please let me know. Similarly, if you have ideas for making the course more accessible, please share them with me.

0.2 Hardware Recommendations

- I strongly recommend *against* tablets (iPad, Surface) for this course
- If you use RStudio Cloud, you have more flexibility for hardware. You can probably get away with using a netbook (Chromebook) if you take this approach.
  - Though see the note below about usage limits for RStudio Cloud.
- Otherwise, at the start of the semester, I recommend that your machine have at least
  - 200 GB free hard drive space
  - 8 GB RAM
  - Intel i5 processor
  - Operating system capable of running R 4.0.0
    * Mac: 10.13 (High Sierra) or more recent
    * Windows: See here or consider just using RStudio Cloud
    * If you're running Linux I'm going to assume you can figure this out yourself; or consider just using RStudio Cloud

0.3 R and RStudio

0.3.1 The Mac Way

You can find some notes on accessibility and RStudio here: https://support.rstudio.com/hc/en-us/articles/360044226673-RStudio-Accessibility-Features.

With this way, R and RStudio are installed and run on your computer, and all of the data files that you work with will also be on your computer. Unless you're running a Chromebook or similar, your computer is almost certainly more powerful than the default instance that RStudio Cloud gives you (see below), and you can use familiar tools to do things like organize folders and preview files. But sometimes particular details about the way your computer is set up can cause R and RStudio to do weird things, and these problems can be very difficult to diagnose and solve.

1. Go to https://cran.r-project.org/. Download and install the latest release of R for your operating system (which should be Mac OS if you're following this way). As of 2020-06-02, the latest release is 4.0.0.

2. Go to `rstudio.com`. Download and install the latest release of RStudio Desktop for your operating system (which should be Mac OS if you're following this way). As of 2020-06-02, the latest release is 1.3.959.

0.3.2 The Cloud Way

(**Note 2017-07-07** Starting in August, RStudio Cloud is moving out of beta and becoming a regular subscription service. Information on pricing is at https://rstudio.cloud/plans/free. We'll have to try it out, but it seems likely that the Free tier won't give you enough compute time ("project hours") for this course. I can subscribe to the Instructor tier, but that has its own limitations. We'll have to see how things go.)

With this way, R and RStudio are installed and run on virtual computers (called *instances*) on a big server somewhere. Each project can have its own instance, meaning that it runs on a nice clean computer and shouldn't have any weirdness. The RStudio Cloud service is currently in beta; it's free to use, but not completely guaranteed to be free of bugs. In order to use the service, you have to be connected to the internet, and you'll have to use the RStudio interface or a terminal window to do things like move files around. Also, the instances aren't very powerful, and so they'll be slow if you try to work with large data sets or do a complex analysis.

1. Go to `rstudio.cloud`. Look for "Sign Up" in the upper right.[1]

0.4 Slack

For Fall 2020, the class Slack channel is at https://datascienceme-ke13588.slack.com. Please use your UC Merced email to sign up.

If you use a screenreader, here are some notes on accessibility and Slack: https://www.marcozehe.de/status-of-the-accessibility-of-slack/.

0.5 GitHub

1. Go to `github.com`. You can create a new account right on the front page. I don't care what email address you use here.
2. You can fill in your profile if you want.
3. You'll send me the email address associated with your Github account in the pre-class survey.

NB If you're using Windows, you'll need to download and install Git.

0.6 Screen Layouts

In this course, a significant amount of time will be spent on *live coding*. This means that I (or whoever's doing the lecturing) will be writing code in real time, and you'll follow along, transcribing the code into RStudio (or whatever) on your local machine and running it at the same time as me.

Live coding is a great way to learn coding practices. But it requires you to have quick access to 3-5 different things:

- My screenshare (so you can see what I'm typing)
- The course Slack (so you can ask for help or other questions, or make comments)
- Possibly a separate way to put up a flag so that I can see immediately that you have a question
- Your own workspace (RStudio or whatever)
- Possibly a separate app for taking notes

In an ordinary classroom, the first three things would all be handled by the classroom environment. In a fully online setting, everything needs to be handled on your computer.

The most effective way to do this is to have multiple displays. This is also the most expensive.

[1] RStudio gives you the option to log in with Google or GitHub. As a general practice, I recommend always keeping your accounts separate. Use a password manager to generate separate, strong passwords for each site. I use LastPass.

This blog post discusses alternatives to multiple displays. In particular, the window layout below fits the screenshare, chat, and your workspace on to a single laptop display. You can also have a separate text file open in RStudio for notes. You might consider having Slack open on your phone — so that you can keep an eye on the chat — as well in the background on your computer — so that you can alt-tab to it to actually type things.

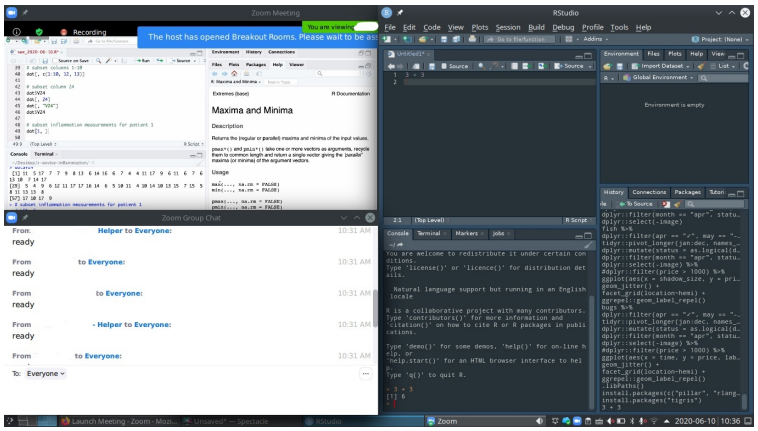

Figure 1: Example of a multiwindow layout for following live coding via Zoom. Source: 'carpentries.org', <https://bit.ly/2YFh9Ea>

0.7 Pre-Class Survey

Please take a few minutes to fill out this survey: https://ucmerced.az1.qualtrics.com/jfe/form/SV_eVU8JWREUu4UNH7.

1 Introduction

1.1 Land Acknowledgement

UC Merced and the City of Merced are on the traditional territory of the Yokut people. This land was stolen by Spanish, Mexican, and American settlers through acts of slavery and genocide. In addition, UC Merced is strongly associated with Ahwahne, known as Yosemite Valley. This valley was the traditional home of the Ahwahnechee people, who were the victims of some especially horrific, state-sponsored genocidal acts. For more on the history of Ahwahne, see https://tinyurl.com/y879jw8s. For more information on land acknowledgments, see https://native-land.ca.

1.2 About me

Dan Hicks is a philosopher turned data scientist turned philosopher.

I use they/them pronouns and identify as nonbinary. I grew up in Placerville, about two hours north of Merced in the Sierra Foothills. One branch of my family came to California during the Gold Rush, so I identify heavily as a Californian and have some complicated feelings about the genocide. I finished my PhD in philosophy of science at Notre Dame in 2012. After that I worked in a series of research positions in academia and the federal government. During 2015-2019 I was using data science methods at least half-time. I joined the faculty at UC Merced in Fall 2019.

- Email: dhicks4@ucmerced.edu
- Student hours: By appointment: https://doodle.com/mm/danhicks/office-hours
- Website: https://dhicks.github.io/

1.3 What this course isn't, and is

Is Not:

- a statistics course (in the way you think)
- a general introduction to software engineering
- a basic introduction to R? (Unless that's what you all need?)
- carefully planned out from beginning to end

Is:

- an introduction to data science
- about data cleaning, data management, and exploratory data analysis
- habituation to some good software engineering practices that are especially valuable for data science work
- in the pre-alpha stage of development

1.4 Learning outcomes

By the end of the course, students will be able to

1. apply concepts from software engineering and philosophy of science to methodological decisions in data science, including
    - functional vs. object-oriented programming paradigms (Chambers 2014)
    - waterfall vs. spiral model of development
    - ladder of models
    - reproducibility vs. robustness vs. replicability
    - value and hazards of open data
    - thick data (Wang 2016)
2. use exploratory data analysis techniques and tools to identify potential data errors and potential phenomena for further analysis
    - checklist (Huebner, Vach, and le Cessie 2016)
    - `skimr`, `visdat`
    - 1D summaries: histograms, density, ecdf plots
    - 2D summaries: scatterplots, boxplot, violin plot, beeswarm, upset plots; pairs plot (https://www.data-imaginist.com/2019/a-flurry-of-facets/); correlation matrix
3. clean raw data and produce a reproducible codebook for both downstream analysis and public release of data according to FAIR standards (Wilkinson et al. 2016)
    - [create and use a cloud database to work with big data]
4. manage data, analysis, and outputs for reproducibility (assembling a project compendium; Gentleman and Lang 2004) using best practices of data handling, a clear directory structure (Wilson et al. 2017), self-documenting code, version control, build automation, [and continuous integration]
5. apply philosophical and data science concepts to integrated ethical-technical analysis to case studies in algorithmic injustice
    - understand the background on the COMPAS case: Angwin et al. (2016); Corbett-Davies et al. (2016)
    - reproduce several fairness statistics on COMPAS data
    - explain tradeoffs between false negatives and false positives in terms of inductive risk
    - explain Kleinberg, Mullainathan, and Raghavan (2016) impossibility result (not necessarily the proof) and show that it applies to the COMPAS datas
    - critically discuss profiling and thresholds (Pierson et al. 2020) as potential explanation for differences in "base rates" of recidivism

- critically locate discussions of fairness and recidivism in a broader social context, and identify implications for working data scientists (Selbst et al. 2018; Hoffmann 2019; Hanna et al. 2019)

1.5 Workflow

- The first ~10 weeks of the course will focus on skill development, with a mix of livecoding lectures and labs.
  - Labs will be set up to give you automated feedback.

  - There will probably be asynchronous discussion using Slack for some of the labs.
- On November 10 we'll be working on project with Hanna Gunn's Data Ethics course.

- The last few weeks of the course will be devoted to term projects.
  - Tentative idea for this: go find some data; clean it, explore it; produce a codebook and reproducible exploratory data analysis

1.6 Notes on some course design decisions

1.6.1 R, not Python

- Python is a perfectly good language for general programming
  - And has some advantages over R in certain data science aspects, including working with strings and developing custom data structures

However,

- R is a better language for doing data science (Shotwell 2019)
  - Vectors, data frames, and common statistical models are installed and loaded by default
  - R is more functional (Wu 2019; Paleologo 2018)
- R is also better for beginners
  - CRAN
  - RStudio

1.6.2 Tidyverse R, not base R

- "Base R" refers to the central code for R and the packages included when you install it
  - Base R is universal and free of additional dependencies. Code written entirely in base R will run on any other R installation (with the same version).
  - However, because the R Core Development Team has always thought backwards-compatibility is important, base R has some idiosyncracies (Keyes 2016)
- `tidyverse` is a suite of packages developed by the manufacturers of RStudio
  - A very useful set of general tools can be loaded simply by calling `library(tidyverse)`
    * And some more specialized tools are also installed as part of the suite
  - Tidyverse is designed to mitigate a lot of the base R idiosyncracies
  - While making code more human-readable and easier to reason about
  - However, several of its tools use something called "non-standard evaluation" (NSE).
    * NSE makes tidyverse code easier to write for beginners, because you don't have to think as much about where exactly your variables are located
    * At the same time, NSE creates problems for intermediate coders, because passing tidyverse-style "bare names" through function arguments causes cryptic errors

1.6.3 Scripts, not notebooks

- Quite a few scientists like the way notebooks mix together code, visualizations, tables, and text (Perkel 2018)

However,

- Notebooks do not play nicely with version control (Ahemad 2018; Whitmore 2015)
- Notebooks encourage bad software engineering habits and discourage good ones (Grus n.d.; Hiltch 2019)

2 What is this thing called data science?

2.1 Standard definition

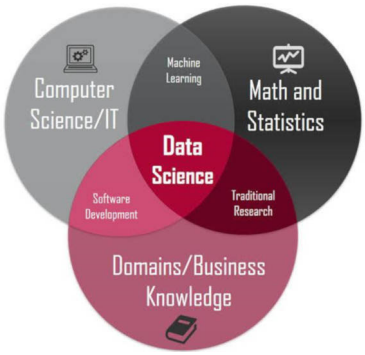

Figure 2: Data science, defined as the intersection of CS, stats, and "business knowledge." Source: <https://www.kdnuggets.com/2020/08/top-10-lists-data-science.html>

- The intersection of computer science/software engineering, statistics, and "business knowledge"

Or, more concretely,

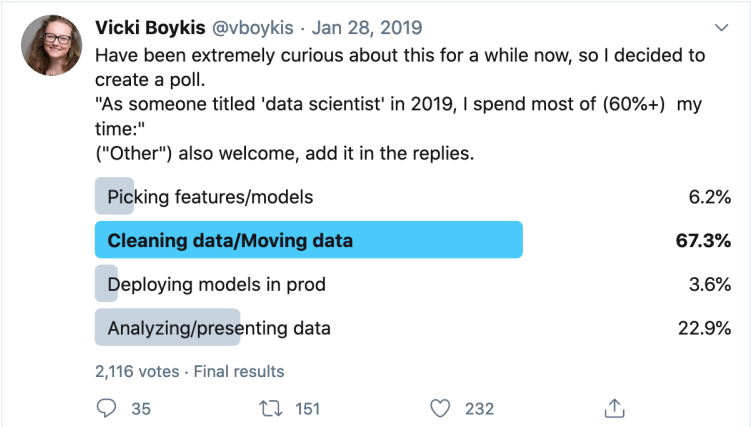

Figure 3: Boykis Twitter poll of "data scientists"; 67% said they spend most of their time cleaning data/moving data. Source: <https://twitter.com/vboykis/status/1089920316644704256>

However,

- This focuses on *tools and techniques*, not the epistemic and practical *goals* of data science
  - An ecologist spends most of their time collecting specimens in the field and processing them in a lab, vs.
  - An ecologist studies interactions among organisms and their environment

2.2 A prior question: What is this thing called "data"?

- *Representational view*: Data are "reliable representations of reality which are produced via the interaction between humans and the world"

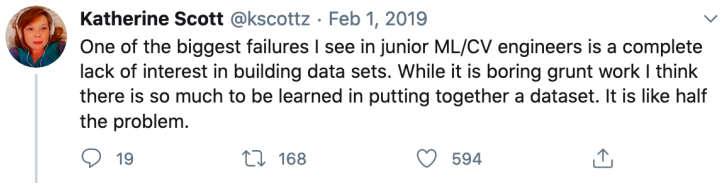

Figure 4: Scott tweet: "One of the biggest failures I see in junior ML/CV engineers is a complete lack of interest in building data sets. While it is boring grunt work I think there is so much to be learned in putting together a dataset. It is like half the problem." Source: <https://twitter.com/kscottz/status/1091423467772162049>

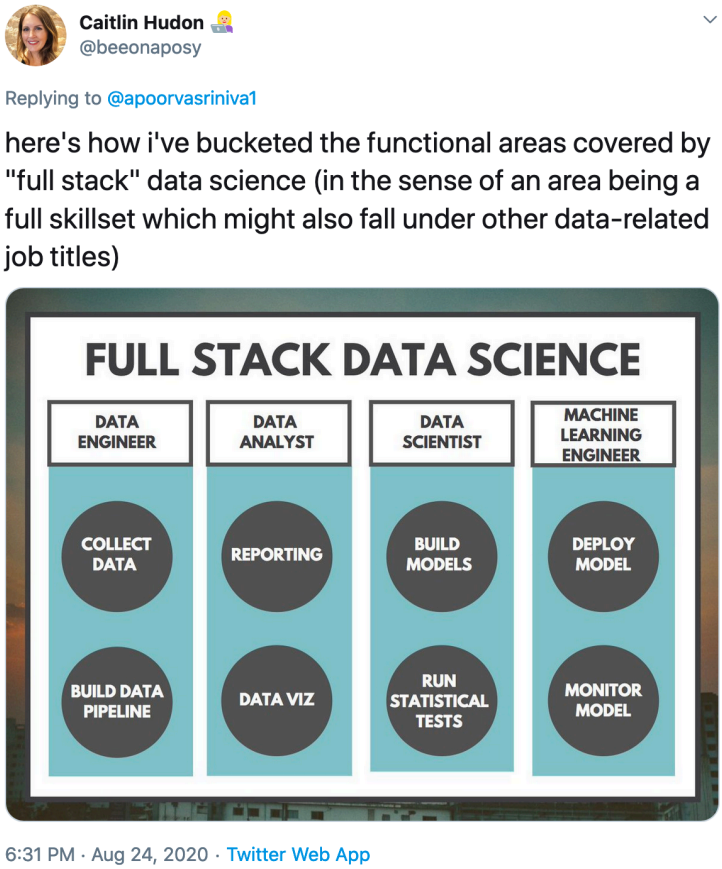

Figure 5: Hudon tweet containing a graphic labelled "Full Stack Data Science," with categories "data engineer" (entries collect, build data pipeline), "data analyst" (entries reporting, data viz), "data scientist" (entries build models, run statistical tests), and "machine learning engineer" (entries deploy model, build model). Source: <https://twitter.com/beeonaposy/status/1298070348844457987>

  - "The production of data is equivalent to 'capturing' features of the world that can be used for systematic study"
  - "Data [are seen as] as 'raw' products of research, which are as close as it gets to unmediated knowledge of reality"
  - Data provide "an objective foundation for the acquisition of knowledge" (Leonelli 2020)
  - **Data become data when they're produced according to a reliable, standardized measurement process**
- *Relational view*: "Data are objects that are treated as potential or actual evidence for scientific claims in ways that can, at least in principle, be scrutinised and accounted for"
  - "Any object can be used as a datum, or stop being used as such, depending on the circumstances"
  - **Data become data when they're used as evidence** (Leonelli 2020)
  - Note implications for ideas of "unmediated knowledge" and "objective foundation"

2.3 Back to "data science"

- Data science lab science (often)
  - Data collection isn't standardized
    * Found (or bought), not made
    * Data as byproduct
    * This is the point of the tweets above
    * ex: scraping from Twitter, Wikipedia, administrative records
  - Remixing and recombining data
    * ex: pesticide use + Census data in Hicks (2020)
  - Data first, questions later
  - **Relational view is often a better fit**
- The goal of data science is to **produce data**
  - To create, transform, and juxtapose objects such that they can be used as evidence
  - Can be done experimentally in a lab, but in other ways too

Part I

Some fundamental tools

3 Git and Version Control

3.1 Some motivating examples

- While working on your analysis code, you accidentally delete the first 35 lines of the script. You only discover this three days later, when you restart R and try to run the script from the top. Because of the time you last half of your senior thesis, you've been saving every couple of minutes.

- You're working on a paper with two coauthors. You prepare the final draft to send for submission: `paper final.docx`. But one of your coauthors discovers a typo. Now it's `paper final fixed typo.docx`. Another realizes six references are missing. `paper final fixed typo refs.docx`. That's getting confusing so you change it to `paper 2 Aug 2021.docx`. Once it comes back from review you need to make revisions. Now you have `paper 30 Jan 2022.docx` and, after your collaborators make their changes, `paper 12 February 2022 DJH.docx` and `paper 12 February 20222 final.docx`.
- You have a complicated analysis spread over several scripts. You want to explore a variant analysis, but doing so will involve changes in 15 different places across 3 different files. You're not sure if this variant analysis will work; you may or may not want to keep it.

3.2 Version control

- Basic idea: Tools for tracking and reversing changes to code over time
- Useful for identifying and reversing breaking changes
- Implementations upload to cloud, track who contributes code, control who can suggest vs. actually change code
- Good for collaboration, publishing code
- git
  - One of many version control systems
  - Very popular in part thanks to GitHub, which provides free hosting for open-source projects

3.3 How to do things with git

- Getting help
  - `git command -h`
  - `man git-command` (hit `q` to exit)
- Interfaces
  - Command line
  - RStudio projects
  - GUIs: GitHub, Sourcetree
- Setup
  - Installation
  - User config

```
$ git config --global user.name "Maria Hernandez"
$ git config --global user.email mhernandez123@ucmerced.edu
$ git config --global core.editor "nano"
```

  - You don't have to use `nano`, but it's the best option for beginners. For more on text editor options, see here https://git-scm.com/book/en/v2/Appendix-C%3A-Git-Commands-Setup-and-Config#_core_editor
- `git init`
  - Create a folder called `test` and open a terminal window there.

```
$ git status
$ git init
$ git status
```

- `add` and `commit` a file Create a text file and save it as `readme.txt` in `test`.

```
$ git status
$ git add test.txt
$ git status
$ git commit -m "my first commit"
$ git status
```

After adding a file, you can add all changes to tracked files using `-u`:

```
$ git status
$ git add -u
$ git status
$ git commit -m "-u is a shortcut"
```

NB You always need to `add` and then `commit`. Doing just one by itself doesn't do much.

- `.gitignore`

  - Create a subfolder `images` and put an image file of some kind in it.
  - Create a text file `.gitignore` in `test`. Put the following line in:

```
images
```

  - Add another image file. Edit `.gitignore` so that it ignores the first image file but not the second.

  - Track `.gitignore`

3.4 Working with GitHub remotes

- *remote*: A copy of a repository that lives on a server somewhere else

3.4.1 Working with *your own* repos

- On GitHub, click "New" and walk through the steps to create a new repository. Let's call it `mytest`.
- Copy the URL: `https://github.com/username/mytest`
- Back in the Terminal window:

```
$ git remote add origin https://github.com/username/mytest
```

You'll need to authenticate with your GitHub password. If you forget the exact incantation, try `git push`; it will give you a helpful example.

- `push` and then refresh GitHub in your browser

```
$ git push
```

- Edit the repository (eg, use the prompt near the bottom to create a readme). Then:

```
$ git fetch  ## Downloads a comparison, but doesn't change your local files yet
$ git status
$ git pull   ## Changes your local files
$ git status
```

3.4.2 Working with *someone else's* repos

GitHub lets you download someone else's repo ("clone"), and modify it locally, but *not* upload directly. You can suggest a change to someone else's code by submitting a *pull request*, which requires *forking* the repository.

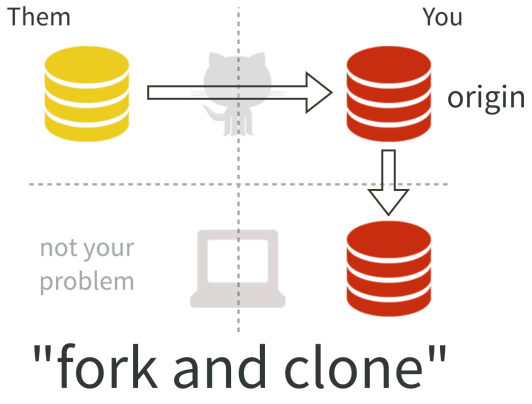

Figure 6: *Forking* copies a repository to your GitHub account. Then you *clone* the copy to your local machine. You can *push* to your remote copy as usual. You can suggest changes to the original using a *pull request*. Source: <https://happygitwithr.com/fork-and-clone.html>

- A demonstration repository: https://github.com/data-science-methods/git-demo

- *Fork step*: Look for the `fork` button in the upper-right

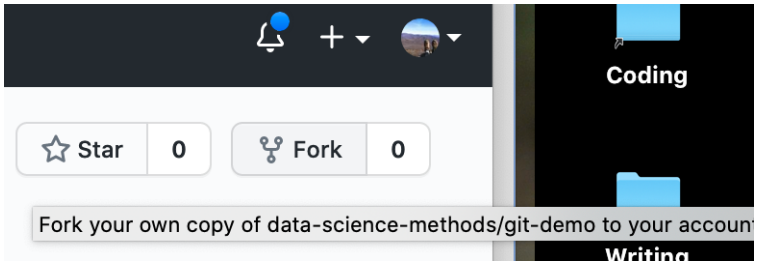

Figure 7: The *fork* button is near the upper-right corner of a GitHub repository page. I wasn't able to find a keyboard shortcut for this. :-(

- *Clone step*: After creating the fork, you need to download a copy to your machine. Open a Terminal window to the folder *right above* where you want the repository folder to be located, eg, `projects`.

```
$ git clone https://github.com/YourUsername/git-demo
$ cd git-demo
$ git status
```

- *Push step*: Make changes locally, then push them back up to *your* GitHub copy of the repo
- *Pull request*: *[todo]*

3.5 Moar git stuff

[todo] - browsing history on GitHub - stretch goals: `branch`; `checkout`

4 R Basics

4.1 Let's get started with RStudio

- Quick tour of RStudio's UI
- Let's plot a thing (below)
- Some idiosyncracies of my code
  - `'` rather than `"`
  - `##` for documentation comment
  - `=` not `<-` for assignment
- Essential Global Options
  - Never save .Rdata, load workspace
- Super useful keyboard commands:
  - Insert a pipe: +Shift+m
  - Comment/uncomment: +Shift+c
  - Tools → Modify Keyboard Shortcuts…
    - +1, +2, +3
  - Cheatsheet: https://raw.githubusercontent.com/rstudio/cheatsheets/master/rstudio-ide.pdf
- **If you're already familiar with the basics of R**, I recommend spending some time with Wickham (2014).

```r
library(tidyverse)
library(gapminder)
library(plotly)

data(gapminder) ## Gets a dataset from a loaded package
```

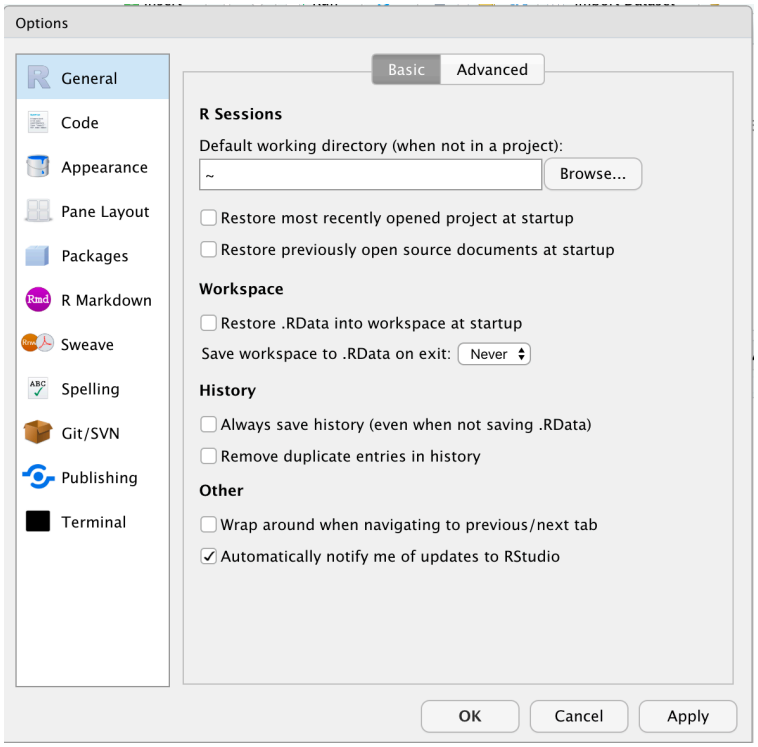

Figure 8: Recommended RStudio configuration

```
gapminder
?gapminder

ggplot(data = gapminder, aes(x = gdpPercap, y = lifeExp, color = continent,
                             size = log10(pop))) +
  geom_point() +
  facet_wrap(vars(continent))

ggplot(data = gapminder, aes(x = year, y = lifeExp, color = continent)) +
  geom_line(aes(group = country)) +
  facet_wrap(vars(continent))

ggplotly()
```

4.1.1 Exercise

- Moar plots
- Selecting variables, and some common math functions

```
gapminder$lifeExp
## These are useful in certain circumstances, but harder to read
gapminder[['lifeExp']]
gapminder[,'lifeExp']
gapminder[,4]

mean(gapminder$lifeExp)
sd(gapminder$lifeExp)
```

```r
log(gapminder$lifeExp)
gapminder$lifeExp / 100

cor(gapminder$lifeExp, gapminder$year)
```

4.1.2 Exercise

Calculate correlations between life expectancy, population, and GDP per capita

4.2 Base R data types

4.2.1 Numerics

```r
foo = 2
bar = 3
foobar = foo + bar
foo / bar
try(foo / 0)
```

4.2.2 Vectors

```r
foo = c(2, 3, 4)
bar = c(5, 6, 7)
foo[2]
foo[2:3]
try(foo[1, 3])
foo[c(1, 3)]

foo + bar
foo * bar
## NB difference between vector and length-1 comparisons
bar == foo + 3
identical(bar, foo + 3)

c(foo, bar)
```

4.2.3 Strings

```r
foo = c('monkey', 'banana')
bar = c('house')
## How to put these together?
try(foo + bar)
c(foo, bar)
paste(foo, bar)
paste0(foo, bar)
```

4.2.4 Factors

```r
foo_str = c('foo', 'foo', 'bar', 'zoo')
as.integer(foo_str)
foo_fct = as.factor(foo_str)
```

```r
as.integer(foo)
levels(foo_fct)
```

4.2.5 Lists

```r
some_list = list('foo', 'bar', 12)
some_list
some_list[2]

nested_list = list('foo', 'bar', some_list)
nested_list
nested_list[[3]]

moar_list = list('foo' = 12, 'bar' = 13)
moar_list$foo
```

4.2.6 Data frames

- `gapminder` is a data frame
- Technical point: a data frame is a list
  - "Columns" = list elements
  - Restriction: all of the elements must have the same length
- A tibble (`tbl_df`) is a data frame with nice printing (part of the tidyverse)
  - Plus some technical restrictions that make it nicer to work with

Like 99% of the time you create a data frame by loading data or manipulating another data frame. But it's good to know how to create them by hand:

```r
foo = 1:5
bar = letters[1:5]

data.frame(foo, bar)
tibble(foo, bar)

## tribble() is super useful when you're creating a dataframe entirely by hand
tribble(~foo, ~bar,
        1, 'a',
        2, 'b',
        3, 'c',
        4, 'd',
        5, 'e')
```

4.3 Examining variables

- `length()`, `dim()`
- `str()`
- `class()`
- `head()`
- challenge

4.3.1 Exercise

- What's the difference between `gapminder['country']` (single brackets) and `gapminder[['country']]` (double brackets)?

4.4 Logical operators

```
if (TRUE) {
  print('it\'s true!')
} else {
  print('it\'s false!')
}

foo = c(1, -1, -3)
foo > 0
if (foo > 0) {
  print('greater than zero!')
}

if (any(foo > 0)) {
  print('at least one is greater than zero!')
}
if (all(foo > 0)) {
  print('all are greater than zero!')
} else {
  print('at least one is not greater than zero!')
}

if (foo > 0 | foo < 0) {
  print('non-zero')
}
if (foo > 0 || foo < 0) {
  print('non-zero')
}

if (foo != 0) {
  print('non-zero')
}
if (all(foo != 0)) {
  print('non-zero')
}

bar = c(17, 25, 0, -14)
ifelse(bar > 0, 'positive', 'negative or zero')
case_when(bar > 0  ~ 'positive',
          bar == 0 ~ 'zero',
          bar < 0  ~ 'negative')
```

4.5 Missing data

https://gge-ucd.github.io/R-DAVIS/lesson_how_r_thinks_about_data.html#missing_data

```
missing = c(7, 17, NA, 12)
missing
mean(missing)
mean(missing, na.rm = TRUE)
is.na(missing)
```

4.5.1 Exercise

- What do the logical operators do with `NA`s?

5 Functional Programming and Data Science

- Recommended reading: Chambers (2014); parts II and III of Wickham (2014)
- Programming paradigms
  - *Procedural*: Software is a series of instructions ("procedures"), which the computer carries out in order. Special instructions (if-then, loops) are used to change the order based on inputs or other conditions.
  - *Object-oriented*: Software is made up of objects, which have properties ("attributes," including other objects) and do things ("methods").
    - Example: A board game consists of 1 or more players, a board, and a deck of cards.
      - Players each have a hand of cards (attribute), can draw cards from the deck into their hand (method, which also changes the deck's attributes), and can play cards to manipulate tokens on the board (method, which also changes the board's attributes).
      - The game itself is an object, with the players, board, and deck for its attributes and methods to things up, run turns in sequence, determine the winner.
      - Separate objects might be responsible for displaying the current state of the game and getting input from the users.
    - Much better than procedural paradigms for *encapsulation*: When a player draws a card from the deck, we just need to know the inputs and outputs for `Deck.draw()`, not the details of how the deck tracks the order of cards and which ones have already been drawn.
    - Much easier to reason about systems with lots of dynamic, interacting parts.
    - Examples (mostly): Python, C++, Java
  - *Functional*: Software is made up of functions, which are run sequentially on the inputs
    - Example: Raw data are read in, cleaned, and then a regression model is fit. The model coefficients are tested for statistical significance, and a table and plot are written out to disk.
    - **Data analysis as a building series of pipes**
    - Examples (mostly): R, Lisp, Haskell
  - A couple of extra rules for functional programming
    - *Immutability*: Once the value is assigned to a variable, that value cannot be changed
    - *No side effects*: Functions don't change anything outside of themselves; they only return a value

6 Warnings, Errors, and Getting Help

6.1 Recommended reading and watching

- https://stackoverflow.com/a/5963610/3187973
  - https://github.com/jennybc/debugging#video

6.2 Messages, warnings, and errors

- *Message*: Things are fine, but here's some information you should know
- *Warning*: Uhhhh I'm gonna keep going, but maybe this isn't what you want
- *Error*: Nope. I'm stopping here. You need to fix the thing.

6.3 Where to go for help

- Isolate the problem
- Restart your session: Session → Restart R
- Local help: `?fun`
- StackOverflow: https://stackoverflow.com/questions/tagged/r
  - CRAN → BugReports (usually GitHub Issues)

```r
library(lubridate)

some_data = c('June 2002', 'May 15, 2007', 'August 2007')
mdy(some_data)
parse_date_time(some_data, orders = 'mdY')
```

Let's try SO: https://stackoverflow.com/search?q=%5BR%5D+lubridate+month-year

```r
parse_date_time(some_data, orders = c('mY', 'mdY'))
```

Here's another one:

```
```r
more_data = c('May 7, 2017', 'May 19, 2017', 'May Fifth, 2017')
mdy(more_data)
```
```

SO doesn't seem so helpful: https://stackoverflow.com/search?q=%5BR%5D+lubridate+written+days

Let's check the CRAN page for `lubridate`: https://cran.r-project.org/web/packages/lubridate/index.html

```
![Screenshot of `lubridate` on CRAN, highlighting the BugReports field](images/lubridate_cran.png){ wid
```

Trying a couple of searches gives us a promising result: https://github.com/tidyverse/lubridate/issues?q=is%3Aissue+is%3Aopen+mdy

```
![Screenshot of `lubridate` issues page, showing a relevant search result](images/lubridate_issues.png)
```

This is a known bug; it looks like they're thinking about doing something about it, but the only workaround is to create an NA: https://github.com/tidyverse/lubridate/issues/685

6.4 Writing a reproducible example: `reprex`

- https://reprex.tidyverse.org/
- https://reprex.tidyverse.org/articles/articles/learn-reprex.html
- https://reprex.tidyverse.org/articles/reprex-dos-and-donts.html

6.5 Do not do these things in your reprex

If you do them, Jenny Bryan will come to your office and set your computer on fire.

- `setwd('/users/danhicks/projects/catsaregreat/myscript/')`
  - Used to ensure that R is running where your file is
  - Unnecessary if you're opening different projects in different RStudio sessions
  - Will cause irrelevant errors on any other system
  - Aside: use `file.path()` or `here::here()` to build paths

- `rm(list=ls())`
  - Used because people think it clears out the global environment
  - Unnecessary if you're regularly using Session → Restart R
  - Also unnecessary at the top of a `Rmd` file, which is always knit in a new session
  - Doesn't actually clear out the global environment
    * eg, doesn't unload packages or reset `options()`

Not on Bryan's list, but a bonus thing to not do:

- `require(package)`
  - If `package` is installed, will act just like `library()`
  - If not, will return `FALSE`
    * which generally means the script will keep going until you get some kind of error about a missing function sometime later
    * probably not the error you wanted help with
    * annoying to debug because I have no idea where the function is supposed to come from
  - If `library()` can't find the package, it immediately raises an error
    * I can tell right away what package needs to be installed

Part II

Exploratory Data Analysis

7 Waterfalls, Spirals, and Philosophy of Science

- Waterfall model of development: https://en.wikipedia.org/wiki/Waterfall_model
- Spiral model of development (Boehm 1988)
- "The Scientific Method": https://plato.stanford.edu/entries/scientific-method/#MetMetEndMet

8 Exploratory Data Analysis and the Ladder of Models

Part III

Cleaning Data

Part IV

Reproducibility

Part V

Case Study: The COMPAS Dataset

Ahemad, Faizan. 2018. "Version Control with Jupyter Notebooks." Medium. October 10, 2018. https://towardsdatascience.com/version-control-with-jupyter-notebooks-f096f4d7035a.

Angwin, Julia, Jeff Larson, Surya Mattu, and Lauren Kirchner. 2016. "Machine Bias: There's Software Used Across the Country to Predict Future Criminals. And It's Biased Against Blacks." ProPublica. May 23, 2016. https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing.

Boehm, B. W. 1988. "A Spiral Model of Software Development and Enhancement." *Computer* 21 (5): 61–72. https://doi.org/10.1109/2.59.

Chambers, John M. 2014. "Object-Oriented Programming, Functional Programming and R." *Statistical Science* 29 (2): 167–80. https://doi.org/10.1214/13-STS452.

Corbett-Davies, Sam, Emma Pierson, Avi Feller, and Sharad Goel. 2016. "A Computer Program Used for Bail and Sentencing Decisions Was Labeled Biased Against Blacks. It's Actually Not That Clear." Washington Post. October 17, 2016. https://www.washingtonpost.com/news/monkey-cage/wp/2016/10/17/can-an-algorithm-be-racist-our-analysis-is-more-cautious-than-propublicas/.

Gentleman, Robert, and Duncan Temple Lang. 2004. "Statistical Analyses and Reproducible Research." *Bioconductor Project Working Papers*, May. https://biostats.bepress.com/bioconductor/paper2.

Grus, Joel. n.d. "I Don't Like Notebooks - Joel Grus - #JupyterCon 2018." Google Docs. Accessed May 6, 2020. https://docs.google.com/presentation/d/1n2RlMdmv1p25Xy5thJUhkKGvjtV-dkAIsUXP-AL4ffI/edit?usp=embed_facebook.

Hanna, Alex, Emily Denton, Andrew Smart, and Jamila Smith-Loud. 2019. "Towards a Critical Race Methodology in Algorithmic Fairness," December. https://doi.org/10.1145/3351095.3372826.

Hicks, Daniel J. 2020. "Census Demographics and Chlorpyrifos Use in California's Central Valley, 2011–15: A Distributional Environmental Justice Analysis." *International Journal of Environmental Research and Public Health* 17 (7, 7): 2593. https://doi.org/10.3390/ijerph17072593.

Hiltch, Or. 2019. "Jupyter Notebook Is the Cancer of ML Engineering." Medium. October 17, 2019. https://medium.com/@_orcaman/jupyter-notebook-is-the-cancer-of-ml-engineering-70b98685ee71.

Hoffmann, Anna Lauren. 2019. "Where Fairness Fails: Data, Algorithms, and the Limits of Antidiscrimination Discourse." *Information, Communication & Society* 22 (7): 900–915. https://doi.org/10.1080/1369118X.2019.1573912.

Huebner, Marianne, Werner Vach, and Saskia le Cessie. 2016. "A Systematic Approach to Initial Data Analysis Is Good Research Practice." *The Journal of Thoracic and Cardiovascular Surgery* 151 (1): 25–27. https://doi.org/10.1016/j.jtcvs.2015.09.085.

Keyes, Os. 2016. "Rbitrary Standards." March 12, 2016. https://ironholds.org/projects/rbitrary/.

Kleinberg, Jon, Sendhil Mullainathan, and Manish Raghavan. 2016. "Inherent Trade-Offs in the Fair Determination of Risk Scores," November. http://arxiv.org/abs/1609.05807.

Leonelli, Sabina. 2020. "Scientific Research and Big Data." In *The Stanford Encyclopedia of Philosophy*, edited by Edward N. Zalta, Summer 2020. Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/archives/sum2020/entries/science-big-data/.

Paleologo, Giuseppe. 2018. "What's the Difference Between Machine Learning in Python and R?" February 28, 2018. https://www.quora.com/Whats-the-difference-between-machine-learning-in-Python-and-R/answer/Giuseppe-Paleologo.

Perkel, Jeffrey M. 2018. "Why Jupyter Is Data Scientists' Computational Notebook of Choice." *Nature* 563 (October): 145. https://doi.org/10.1038/d41586-018-07196-1.

Pierson, Emma, Camelia Simoiu, Jan Overgoor, Sam Corbett-Davies, Daniel Jenson, Amy Shoemaker, Vignesh Ramachandran, et al. 2020. "A Large-Scale Analysis of Racial Disparities in Police Stops Across the United States." *Nature Human Behaviour*, May, 1–10. https://doi.org/10.1038/s41562-020-0858-1.

Selbst, Andrew D., Danah Boyd, Sorelle Friedler, Suresh Venkatasubramanian, and Janet Vertesi. 2018. "Fairness and Abstraction in Sociotechnical Systems." SSRN Scholarly Paper ID 3265913. Rochester, NY: Social Science Research Network. https://papers.ssrn.com/abstract=3265913.

Shotwell, Gordon. 2019. "Why I Use R." Title. December 30, 2019. https://blog.shotwell.ca/posts/why_i_use_r/.

Wang, Tricia. 2016. "Why Big Data Needs Thick Data." Medium. January 20, 2016. https://medium.com/ethnography-matters/why-big-data-needs-thick-data-b4b3e75e3d7#.ur5gl84nj.

Whitmore, Jonathan. 2015. "Jupyter Notebook Best Practices for Data Science." Silicon Valley Data Science. September 10, 2015. https://www.svds.com/jupyter-notebook-best-practices-for-data-science/.

Wickham, Hadley. 2014. *Advanced R*. CRC Press. https://adv-r.hadley.nz.

Wilkinson, Mark D., Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, et al. 2016. "The FAIR Guiding Principles for Scientific Data Management and Stewardship." *Scientific Data* 3 (1, 1): 1–9. https://doi.org/10.1038/sdata.2016.18.

Wilson, Greg, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. 2017. "Good Enough Practices in Scientific Computing." *PLOS Computational Biology* 13 (6): e1005510. https://doi.org/10.1371/journal.pcbi.1005510.

Wu, Jun. 2019. "Python Vs. R — Choosing the Best Programming Language for Data Science." Medium. June 25, 2019. https://towardsdatascience.com/python-vs-r-choosing-the-best-programming-languages-for-data-science-b1327f01f6bf.